

Simultaneous-Move Search and Guided Monte Carlo Tree Search for Turn-Based Pokémon Battle AI

Clement Lai, Pearly Lin, Ng Qiting, Cheryl Choo, Novelie Voon

1 Introduction

Pokémon battles look like an easy target for classic game-tree search as each side has a handful of moves (Eg. Cut, Fly), the state (Eg. Which Pokemon each player has on field, the Pokemon's remaining health point) is mostly visible and the rules are fixed. However, both trainers must select a move before they can view their opponent's move. This breaks the assumption for both Minimax algorithm and Alpha-Beta pruning as they require players to alternate turns and react to the opponent's moves. If these algorithms are applied to Pokémon combats, the tree will model the moves incorrectly and provide information that it should not have to the second player. This results in unfairness during gameplay as the second player will receive an advantage where they know what the move the first player chose and can change their moves accordingly.

This article walks through a small C++ battle simulator built around Pokémon Generation III mechanics, along with three agents with increasing levels of complexity that were built to deal with this problem: a reactive Greedy agent, a simultaneous Move Alpha-Beta (SMAB) agent and a Monte Carlo search agent guided by ideas from Information Set Monte Carlo Tree Search (IS-MCTS).

2 Why Sequential Search Breaks Down

Minimax works by having one player pick the action that maximizes their outcome, assuming the other player will respond by minimizing it. However, it only works because the players move in alternating turns and can see each other's choices before responding, whereas in a Pokémon battle, both trainers lock in a choice at the same time, so there is no time for players to react.

Similarly, Alpha-Beta pruning algorithm is an optimized version of Minimax algorithm that also assumes an alternating-turn game. It reduces the number of game states evaluated without affecting the final decision. In this algorithm, it tries to maintain two values throughout the entire search – Alpha and Beta. Alpha value is the best score that the max player can guarantee while Beta is the best score that the min player can guarantee.

Because both algorithms assume that players move one after another, the tree will be able to alternate between max and min nodes. Yet, Pokémon combat requires both players to choose their action simultaneously, the tree cannot represent the game accurately and thus, will not be able to perform properly and accurately. Therefore, sequential search algorithms like Minimax and Alpha-Beta pruning fail when it comes to Pokémon turn-based combat.

3 System Overview

The Pokémon battle simulator developed using C++ is split into three parts, the first is the battle engine called GameState, the second is the front-end that uses Simple and Fast Multimedia Library (SFML) for manual play and visualization, and lastly the agents. The agents do not touch any of the Graphical User Interface (GUI) code and hence everything discussed here runs identically whether a window is open.

The engine implements Pokémon Generation III-style combat using the standard stat formula, speed-based turn order with priority, a full Type Classification chart and Same-Type Attack Bonus (STAB).

Firstly, there is two different formulae for stat calculation – the first is for health points, and the second is for attack, defense, special attack, special defense and speed formula.

For the health point formula, it uses this to calculate how much health the Pokémon has.

$$HP = \left\lfloor \frac{(2 * Base + IV + \lfloor \frac{EV}{4} \rfloor) * Level}{100} \right\rfloor + Level + 1$$

As for the attack, defense, special attack, special defense and speed formula, it uses the formula below.

$$Stat = \left\lfloor \frac{(2 * Base + IV + \lfloor \frac{EV}{4} \rfloor) * Level}{100} \right\rfloor + 5) * Nature$$

Variable Glossary:

- Base: The base stat value of the Pokémon's species
- Individual Value (IV): Hidden 'genetic' value ranging from 0 to 31 for each stat
- Effort Value (EV): Points gained from defeating specific Pokémon
- Level: The Pokémon's current level
- Nature: Multiplier for the calculated stat (eg. Beneficial nature multiplies the stat by 1.1, Hindering nature multiplies by 0.9 and Neutral nature multiplies by 1.0)

There is 2 different type classification in Pokémon – Physical and Special. Physical Types consist of Normal, Fighting, Poison, Ground, Flying, Bug, Rock, Ghost and Steel while Special Types consist of Fire, Water, Electric, Grass, Ice, Psychic, Dragon and Dark. Each attacking type can counter other attacking types and does different amounts of damage. For instance, Fire Pokémon does two times damage towards Grass, Ice, Bug and Steel type while the damage is halved against Fire, Water, Rock and Dragon typed Pokémon. Fire type has no Pokémon that it cannot deal damage towards.

Pokémon battles also include STAB, which is a mechanic that is used to increase the power of a move when the move's type matches the Pokémon's type. When a Pokémon uses a move that has the same type as itself, a damage multiplier is applied to the move, allowing Pokémon to deal more damage.

A `stepExpecti` function is the most important part for searching. Instead of returning one deterministic result for a chosen pair of moves, it returns a list of pairs, branching on whether each move hits or misses. Both search agents lean on this directly to account for accuracy without needing their own separate randomness model.

4 The Greedy Baseline

The Greedy Agent is the control group in this experiment. For each available move, it computes the score with the formula below.

$$\text{Score} = \text{Power} * \text{STAB} * \text{type} - \text{effectiveness} * \text{accuracy}$$

It then picks the highest-scoring move and heals once if its active Pokémon drops below 30% health. It never considers what the opponent might do next turn. It simply reacts to the instant in front of it, making it fast and efficient. Using the Greedy Agent as the baseline provides a fair benchmark for other agents. If a more complex agent cannot consistently outperform this agent, the extra computation and complexity of the advanced agent is not worth it.

5 Simultaneous Move Alpha-Beta (SMAB)

SMAB is an algorithm based on Alpha-Beta pruning that is modified to work in an environment where both players select their moves simultaneously, rather than one after another. It is a recursive function that starts by generating all legal actions a player can take, then continuing to branch out the legal actions following those actions, keeping track of the worst possible outcome. If a branch in the search's worst outcome has a lower score than the current worst outcome, the agent stops searching on that branch. The algorithm returns the choice with the best outcomes once its search is completed.

Although the advantage of SMAB is that it is exhaustive by considering every action, this comes at a cost as its runtime grows quickly as more actions are added to both player sides.

6 Information Set Monte Carlo Search Agent (IS-MCS)

Rather than checking every possible outcome like SMAB, the IS-MCS agent searches for a sample space to approximate the best outcome. In the prototype, the IS-MCS agent assumes the enemy will be playing like a Greedy agent and creates a sample space based on this assumption. The agent then goes through the possible outcomes of the sample space and returns the choice with the best outcomes.

Compared to SMAB, the IS-MCS agent is cheaper to run as it only considers a subset of outcome rather than evaluating every possible outcome. It is also more accurate as it takes into account how a real opponent would think. However, this accuracy heavily depends on its evaluation of the opponent, so if the opponent deviates from its Greedy algorithm, the agent's choice of the best outcome would no longer be accurate.

7 Shared Evaluation Heuristic

Both search agents lean on the same `evaluate()` function.

Evaluation Score

$$= \text{Total Searching Player HP} - \text{Total Opponent HP} \\ + \text{Bonus from spare healing potion}$$

total party HP for the searching player minus the opponent, plus a small bonus for each spare healing potion.

Using raw HP instead of HP percentage was a deliberate choice – it keeps a single high-HP “wall” Pokémon from being undervalued relative to a fragile one sitting at the same percentage of health. The original design called for folding in stat-stage modifiers, remaining type coverage and entry hazards as well; those are still on the to-do list.

8 Benchmarking

When the program runs, there are three options to choose from and compare the different algorithms. The first option is to compare Greedy against Greedy; the second option is to compare Greedy against SMAB and the third option is to compare Greedy against IS-MCS. The number of matches can be set as well to compare the performance of the different algorithms. In this example, the number of matches will be set to 500 to compare the performance of the agents. The minimum Pokémon level set will be 60 and maximum Pokémon level will be set to 61.

Table 1 Average Computational Time in Milliseconds per Match

	Greedy
Greedy	0.48658 ms
SMAB	292.919 ms
IS-MCS	1049.52 ms

Table 2 Number of Wins Based on The Algorithm for 500 Matches

Player 1	No. of Wins	Player 2	No. of Wins	No. of Draws
Greedy	245	Greedy	255	0
SMAB	359	Greedy	141	0
IS-MCS	345	Greedy	155	0

Based on the results obtained after testing the algorithms out of 500 matches, it can be concluded that both SMAB and IS-MCS algorithms outperform Greedy algorithm by 32.3% and 8.58% respectively. While both algorithms beat Greedy algorithm, SMAB performs better than IS-MCS with 94 more matches won against Greedy (117 matches against 23 matches). Although the average computational time taken for a match is significantly longer than IS-MCS with the computational time taken being more than 88.7% slower than IS-MCS, the actual performance of the model makes up for the computational time. If computational time is an important factor, IS-MCS is a potential model to look at whereas if performance is more critical, SMAB should be taken into consideration.

9 Limitations and Future Work

In actual applications of IS-MCTS, when the AI doesn't know something about the opponent, it is supposed to randomly guess a plausible version of that hidden info before running its search. In our code, that function just returns the state unchanged instead of guessing. For now, it isn't handling hidden information at all. Instead, it just treats everything as if it is fully visible.

Actual applications of MCTS build a tree that moves several moves deep. However, this current version only evaluates the immediate next move using UCB1, then does one rollout and stops, no multi-level tree being built which is closer to a single-round decision than a search tree.

The real SMAB solves a linear program to find a mixed strategy. But for our SMAB, it always commits to one specific move based on worst-case comparisons. It is a reasonable simplification because it meant that our agent could in theory be predicted and countered in a way the full algorithm cannot.

To keep the search fast, when our agents are figuring out what the opponent will do, they mostly assume the opponent won't switch Pokémon. A smart real opponent might switch to counter the player and for now our agent cannot see that coming.

10 Conclusion

Sequential search algorithms fall apart the moment a game asks both players to move at once and Pokémon battles do exactly that. Treating each turn as a payoff matrix, as SMAB does and estimating state value through guided rollouts as the MCTS agent does, are both practical ways around that problem. However, neither agent implemented here is a complete textbook version of the algorithm.

11 References

[Browne 12] Browne, C.B., E. Powley, D. Whitehouse, S.M. Lucas, P.I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. 2012. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4(1): 1-43.

[Cowling 12] Cowling, P.I., E.J. Powley, and D. Whitehouse. 2012. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games* 4(2): 120-143.

[Kocsis 06] Kocsis, L., and C. Szepesvári. 2006. Bandit based Monte-Carlo planning. Paper presented at the 17th European Conference on Machine Learning, Berlin.

[Saffidine 12] Saffidine, A., H. Finnsson, and M. Buro. 2012. Alpha-beta pruning for games with simultaneous moves. Paper presented at the Twenty-Sixth AAAI Conference on Artificial Intelligence, Toronto.